

Proyecto base – Examen Recuperación Biblioteca (Spring Boot)

👉 Todos los métodos de **service** y operaciones principales lanzan:

```
throw new UnsupportedOperationException("TODO");
```

Des de implementar:

- Modelo (JPA + Mongo)
- Repositorios
- Servicios (programación defensiva + reglas de negocio)
- **Controladores REST (si procede)**

Cómo ejecutar

```
mvn clean spring-boot:run
```

Endpoints

Los endpoints están definidos en los controllers (ver **controller/**).

La implementación debe respetar rutas y contratos.

Estructura de paquetes

- **com.biblioteca.model** (entidades JPA + documentos Mongo + enums)
- **com.biblioteca.repository** (JPA repositories)
- **com.biblioteca.repository.mongo** (Mongo repositories)
- **com.biblioteca.service** (interfaces)
- **com.biblioteca.service.impl** (implementaciones STUB)
- **com.biblioteca.controller** (endpoints REST)
- **com.biblioteca.dto** (DTOs)

Datos de ejemplo y Mongo con Docker

Este proyecto incluye **datos de ejemplo** para:

- **H2** mediante **src/main/resources/data.sql**
- **MongoDB** mediante **docker-compose.yml** + **mongo-init/01-seed.js**

Arrancar Mongo

Desde la carpeta del proyecto:

```
docker compose up -d
```

- MongoDB: `localhost:27017`
- Mongo Express (UI): `http://localhost:8081`

Arrancar la API

```
mvn clean spring-boot:run
```

Swagger UI

- `http://localhost:8080/swagger-ui`

Nota: la primera vez que se arranca Mongo con volumen nuevo, se insertan los datos de `mongo-init/01-seed.js`.

Contexto general

Este examen evalúa los mismos **resultados de aprendizaje** que el examen original:

- Modelado relacional con **JPA/H2**
- Uso de **MongoDB** como base documental complementaria
- Separación en capas (`model`, `repository`, `service`)
- Programación defensiva
- Lógica de negocio
- Integración de datos de **dos BBDD**

El examen se divide en **DOS NIVELES**:

Nivel	Tipo de relación	Nota máxima
A	1 : N	6,5
B	N : M	10

El alumno **elige SOLO UNA opción**.

OPCIÓN A – NIVEL BÁSICO (1 : N) → NOTA MÁXIMA 6,5

Dominio: Biblioteca

Modelo relacional (H2 / JPA)

Tabla **SOCIO**

Campo	Tipo
id	Long (PK)
nombre	String
email	String
fechaAlta	LocalDate

Tabla **PRESTAMO**

Campo	Tipo
id	Long (PK)
fechaInicio	LocalDate
fechaFin	LocalDate (nullable)
estado	ACTIVO / DEVUELTO
socio_id	FK → SOCIO

Relación:SOCIO **1** — **N** PRESTAMO**Modelo documental**Colección **socio_detalles**

Campo	Tipo
id	ObjectId
socioid	Long
telefono	String
direccion	String
notas	String

Relación lógica por **socioId**.**Funcionalidades obligatorias****Servicios obligatorios**

- **SocioService**
 - crearSocio(...), **0.5 puntos**

- obtenerSocioPorId(id), **0.5 puntos**
- listarSocios(), **0.5 puntos**
- eliminarSocio(id), **0.5 puntos**

- **PrestamoService**

- crearPrestamo(socioid, fechaInicio), **1 puntos**
- devolverPrestamo(prestamoid), **0.5 puntos**
- listarPrestamosActivosPorSocio(socioid), **0.5 puntos**

- **SocioDetallesService**

- obtenerDetallesPorSocioid(socioid), **2.5 puntos**
 - La operación integra (H2 + Mongo)

Reglas de negocio (Nivel A)

- No se permiten valores **null\vacío**, se devuelve un error contrario.
- **fechaInicio** no puede ser futura. **LocalDate.now**
- Un socio no puede eliminarse si tiene préstamos activos

OPCIÓN B – NIVEL AVANZADO (N : M) → NOTA MÁXIMA 10

Dominio: Biblioteca (MISMA TEMÁTICA)

Modelo relacional (H2 / JPA)

Tabla **SOCIO**

(igual que nivel A)

Tabla **LIBRO**

Campo	Tipo
id	Long (PK)
isbn	String (único)
titulo	String
autor	String
anio	int

Tabla **PRESTAMO_LIBRO** (ENTIDAD INTERMEDIA)

Campo	Tipo
id	Long (PK)
socio_id	FK → SOCIO
libro_id	FK → LIBRO
fechaInicio	LocalDate
fechaFin	LocalDate (nullable)
estado	ACTIVO / DEVUELTO

Relación:

SOCIO N — M LIBRO (mediante PRESTAMO_LIBRO)

Modelo documental (MongoDB)

Colección `socio_detalle`s

(igual que Nivel A)

Funcionalidades obligatorias (Nivel B)**Servicios obligatorios**

- **SocioService**
 - crearSocio(...) , **0.5 puntos**
 - obtenerSocioPorId(id), **0.5 puntos**
 - listarSocios(), **0.5 puntos**
 - eliminarSocio(id), **0.5 puntos**
 - **LibroService**
 - crearLibro(...), **0.5 puntos**
 - listarLibros(), **0.5 puntos**
 - obtenerLibroPorIsbn(isbn), **0.5 puntos**
 - eliminarLibro(id), **0.5 puntos**
 - **PrestamoLibroService**
 - prestarLibro(socioId, libroId, fechaInicio), **0.5 puntos**
 - devolverLibro(prestamoId), **0.5 puntos**
 - listarPrestamosActivosPorSocio(socioId), **1 puntos**
 - listarPrestamosActivosPorLibro(libroId), **1 puntos**
 - **SocioDetalleService**
 - obtenerDetallesPorSocioId(socioId), **3 puntos**
 - La operación integra (H2 + Mongo)
-

Reglas de negocio (Nivel B)

- Un libro solo puede tener **un préstamo activo**
 - No se permite borrar un libro con préstamos activos
 - Fechas no futuras. `LocalDate.now`
 - Validación defensiva completa
-

PROYECTO BASE (STUB)

Estructura

```
src/main/java/com/biblioteca/  
├─ model  
├─ repository  
└─ service
```

Importante

👉 **Debes de complementar el código en función de la opción que elijas:**

El código inicialmente tiene incluido:

```
throw new UnsupportedOperationException("TODO");
```

El alumno debe sustituir estas excepciones por código funcional.
